

分布式NoSQL数据库HBase

Grissom | 2025年11月

目录

CONTENTS

- 1 HBase简介
- 2 HBase原理
- 3 HBase部署
- 4 HBase基本用法

1
chapter

HBase简介

➤ HBase概念

- HBase是一个高可靠性、高性能、面向列、可伸缩的分布式**列式数据库**。
- HBase是Google Bigtable的开源实现。
- HBase的目标是存储并处理大型的数据，更具体来说是仅需使用普通的硬件配置，就能够处理由成千上万的行和列所组成的大型数据。
- HBase利用**HDFS**作为其文件存储系统。



➤ HBase适用场景

超大规模数据存储



- ☐ 非/半结构化数据
- ☐ 小对象 ($\leq 10\text{M}$ 的图片 and 音视频)
- ☐ 结构化数据 (面向高并发点查)

实时读写 / 高并发点查



- ☐ 实时数据接入
- ☐ 高并发 (上万)
- ☐ 简单条件的精确查询



2 chapter

Hyperbase原理

- ✓ 数据模型
- ✓ 系统架构
- ✓ 存储机制
- ✓ 读写流程

➤ 示例：学生信息表

• 四维表：Rowkey | 列族 | 列限定符 | 时间戳 ➡ 二维表：Rowkey | 列

Rowkey	BasicInfo			CourseInfo		timestamp
	name	age	telephone	course	score	
rk001	zhangsan					t1
		18				t4
			1380100001			t9
			1590939995			t5
				math		t2
					90	t3
rk002	lisi					t6
		20				t7
			1391129396			t8
				english		t10
					85	t11

数据读取流程



➤ 命名空间 (Namespace)

- 命名空间是对表的逻辑分组，类似于关系数据库中的Database
- 利用命名空间，在多租户场景下可做到更好的资源和数据隔离

➤ 表 (Table)

- HBase以“表”为单位组织数据
- 表由多行组成

➤ 行 (Row)

- 表以“行”为单位组织数据
- 行由一个Rowkey和多个列族组成
- Rowkey
 - 行的主键，用于唯一地标识和定位行
 - 各行按Rowkey的字典序排列
 - 大小64K

➤ 列族 (Column Family)

- 数据按“列族”垂直分组，每一行由若干列族组成，每个列族下可包含多个列
- 列族必须在建表时明确定义，且不能频繁修改，例如：create ‘student’, ‘basicinfo’, ‘courseinfo’
- 列族的数量不能太多，理论上限制在几十个，实际会更少
- 每一行的列族都相同，但列族不一定要包含数据
- 物理上同一列族的数据存储在一起，即列式存储
- 权限控制、存储以及调优都在列族层面进行

➤ 列与列限定符 (Column & Column Qualifier)

- 一个列族可包含多个列，且各列族可包含不同的列
- 列由列族和列限定符唯一指定，列名由列族名和列限定符组成，以“:”分隔
 - 例如：basicinfo列族包含basicinfo:name、basicinfo:age和basicinfo:telephone三列，其中name、age和telephone是列限定符
- 新列可按需动态加入列族

➤ 单元格 (Cell)

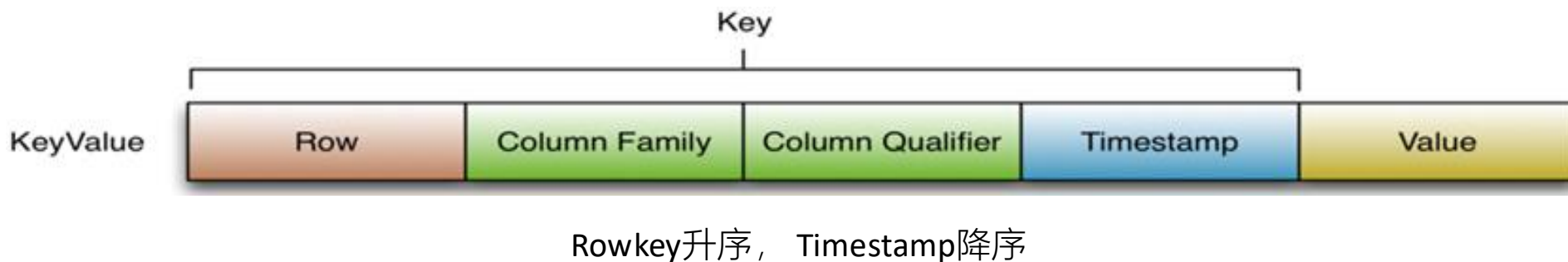
- 单元格由Rowkey、列族、列限定符唯一定位
- 单元格中存放若干<Value, Version>
- 单元格的值是有版本的，版本号和时间戳
- 单元格的值由<Rowkey, Column(Column Family: Column Qualifier), Version>来唯一定位
- 单元格的数据类型为字节数组byte[]

➤ 时间戳 (Timestamp)

- 单元格的值利用时间戳来标识版本，时间戳具有唯一性
- 单元格内不同版本的值按时间倒序排列，最新的数据排在最前面
- 时间戳可以在数据写入时由系统自动赋值（精确到毫秒的当前系统时间），也可以由客户显式赋值，为了避免版本冲突，必须生成具有唯一性的时间戳
- 类型是64位整型

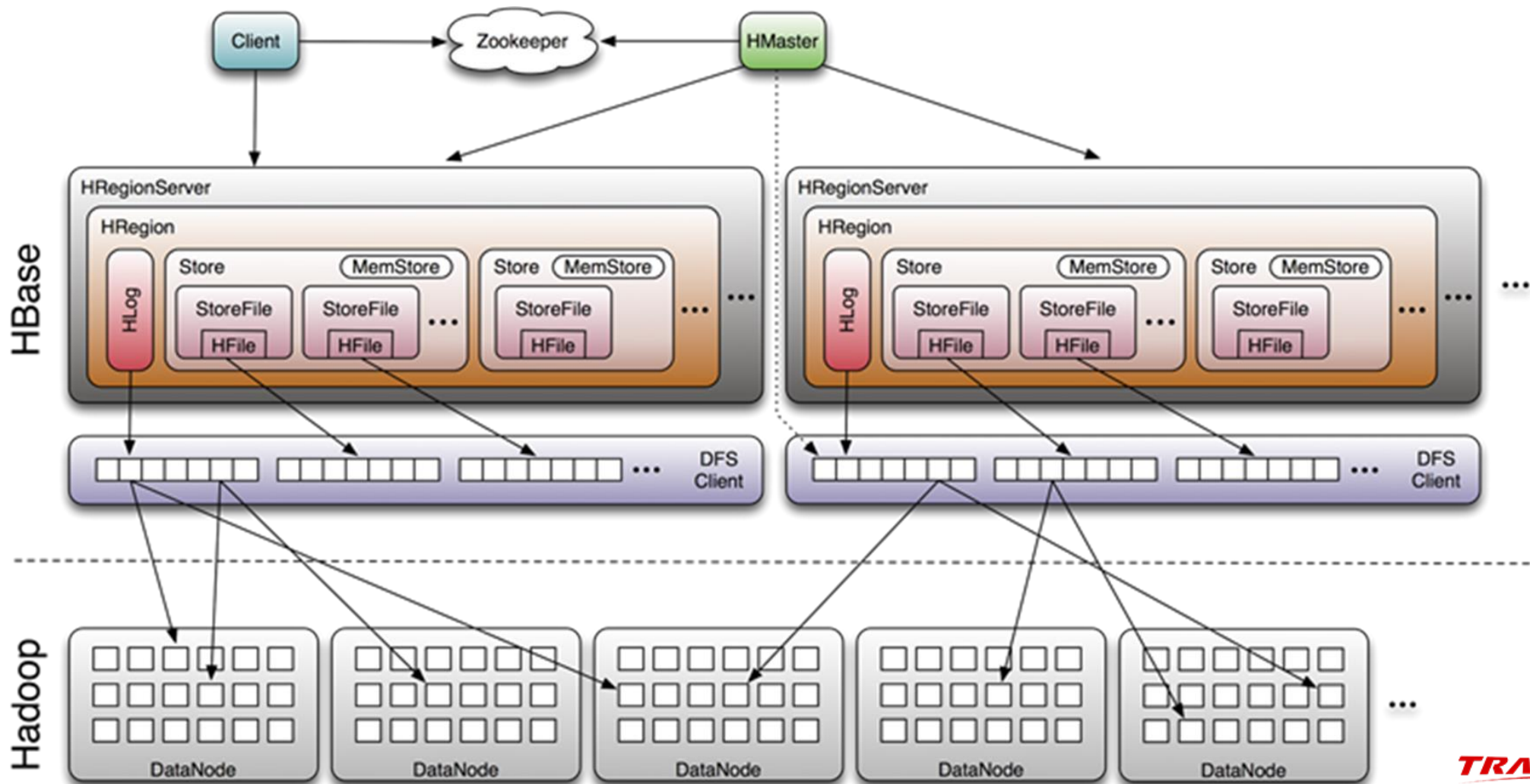
➤ Rowkey & Key

- Rowkey用于唯一标识逻辑上的行
- Key用于唯一标识单元格中的值
 - $\langle \text{Rowkey}, \text{Column family}, \text{Column Qualifier}, \text{Version}, \text{Value} \rangle \rightarrow \langle \text{Key}, \text{Value} \rangle$
 - $\langle \text{Rowkey}, \text{Column family}, \text{Column Qualifier}, \text{Version} \rangle \rightarrow \text{Key}$



➤ 表的特点

- 数据规模大
 - 单表可容纳数十亿行，上百万列
- 无模式
 - 每行可以有任意多的列，列可以动态增加，不同行可以有不同的列，列的类型没有限制
- 面向列族
 - 面向列族的存储和权限控制，支持列族独立查询
- 稀疏
 - 值为空的列不占存储空间，表可以非常稀疏
- 数据多版本
 - 单元格的值可以有多个版本，利用时间戳来标识版本
- 数据无类型
 - 所有数据以字节数组形式存储



➤ HMaster (Master)

- 管理元数据
- 管理表的创建、删除和修改
- 为HRegionServer分配Region
 - HRegionServer宕机后，重新分配其上的Region
- 负责HRegionServer的负载均衡
- 不处理Client的数据读写请求

➤ HRegionServer (Slave)

- 处理Client的数据读写请求
- 管理Region Split（分裂）
- 管理StoreFile Compaction（合并）

➤ Zookeeper

- 实现HMaster高可用
- 监控HRegionServer的上下线信息，并通知HMaster
- 存储元数据的寻址入口
- 存储所有Region的寻址入口

➤ Client

- 通过接口访问Hyperbase
- 为了加快数据访问速度，将元数据、Region位置等信息缓存在Client Cache中

➤ Region

- 分布式存储和负载的最小单元
- 系统将表水平划分（按行）为多个Region，每个Region保存表的一段连续数据
- 默认每张表开始只有一个Region，随着数据不断写入，Region不断增大，当Region大小超过阈值时，当前Region会分裂成两个子Region
- 随着Region的不断增多，HMaster会将部分Region迁移到其他HRegionServer中，实现负载均衡
- 表通常被保存在多个HRegionServer的多个Region中

➤ Store

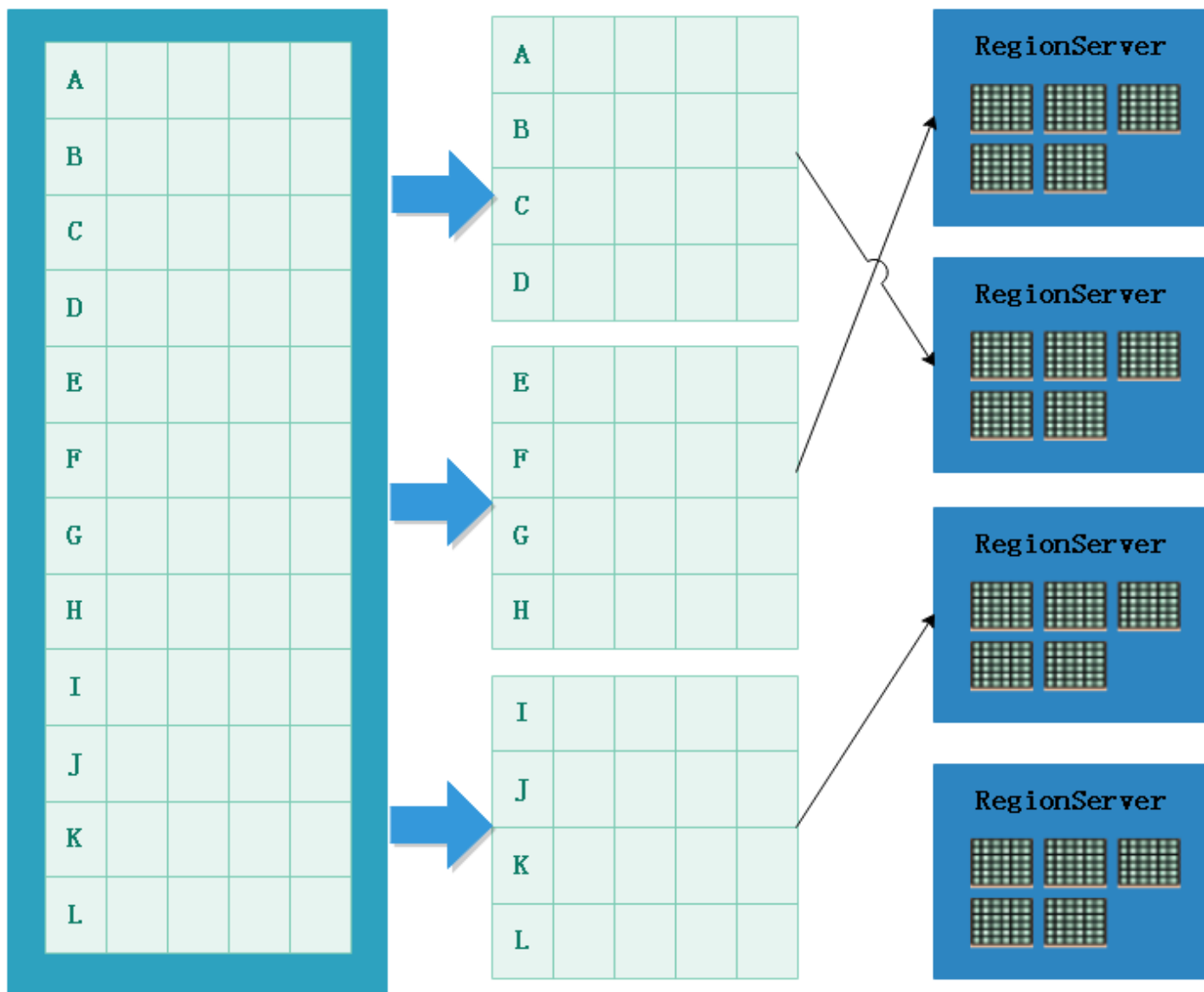
- 一个Region由多个Store组成，每个Store存储一个列族
- Store由内存中的MemStore和磁盘中的若干StoreFile组成

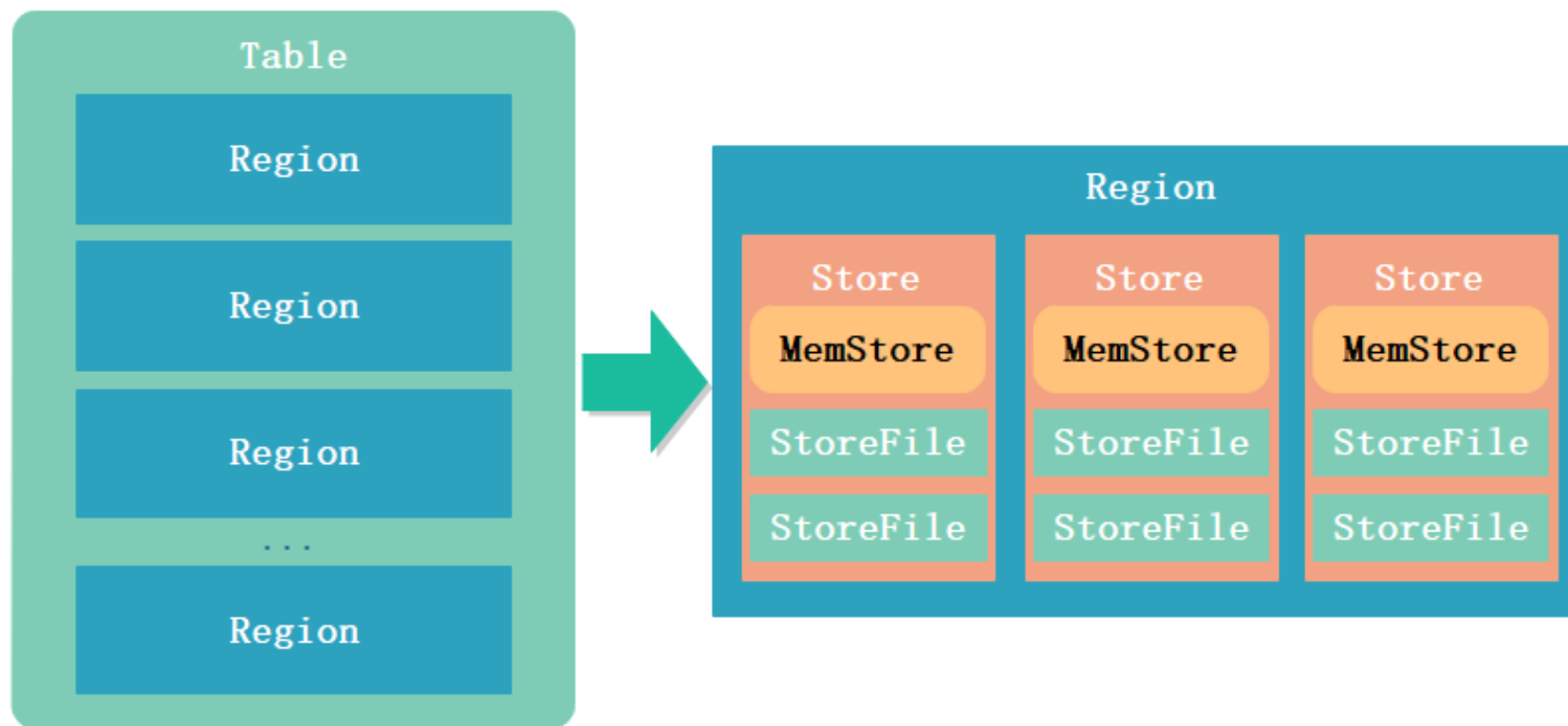
➤ MemStore与StoreFile

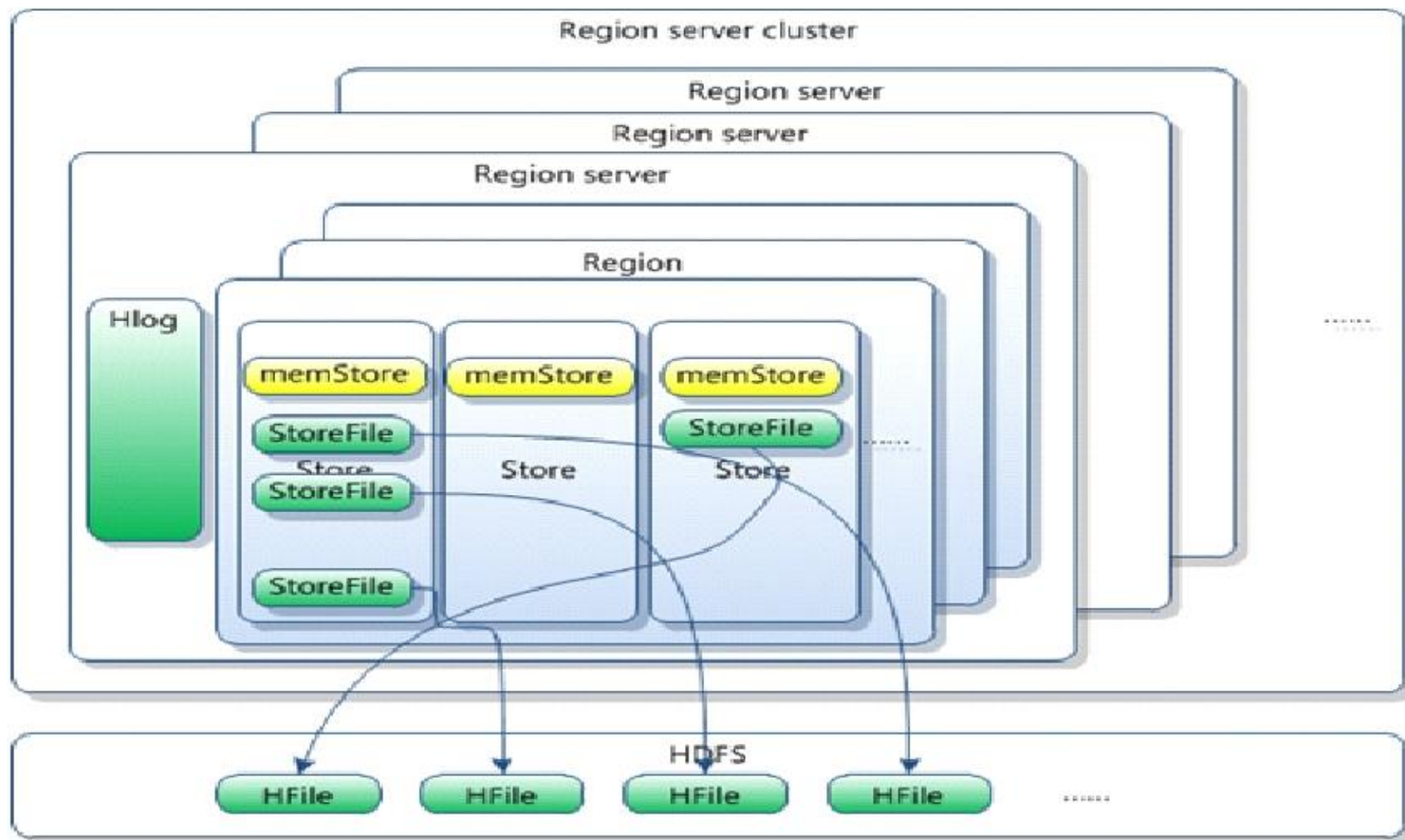
- MemStore是Store的内存缓冲区，数据读写都先访问MemStore
- StoreFile是MemStore的磁盘溢写文件，在HDFS中被称为HFile
- 写数据时，先写MemStore，当数据量超过阈值时，HRegionServer会将MemStore中的数据溢写磁盘，每次溢写都生成一个独立的StoreFile（HFile）
- 当Store中的StoreFile数量超过阈值时，HRegionServer会将若干小StoreFile合并为一个大的StoreFile
- 当Region中最大Store的大小超过阈值时，HRegionServer会将其等分为两个子Region
- Client读取数据时，先找MemStore，再找StoreFile

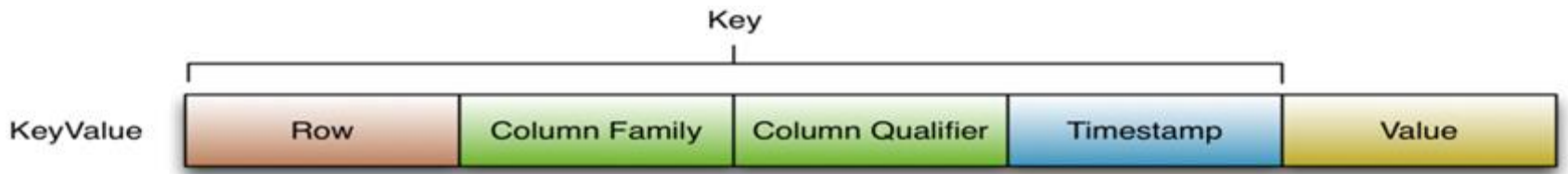
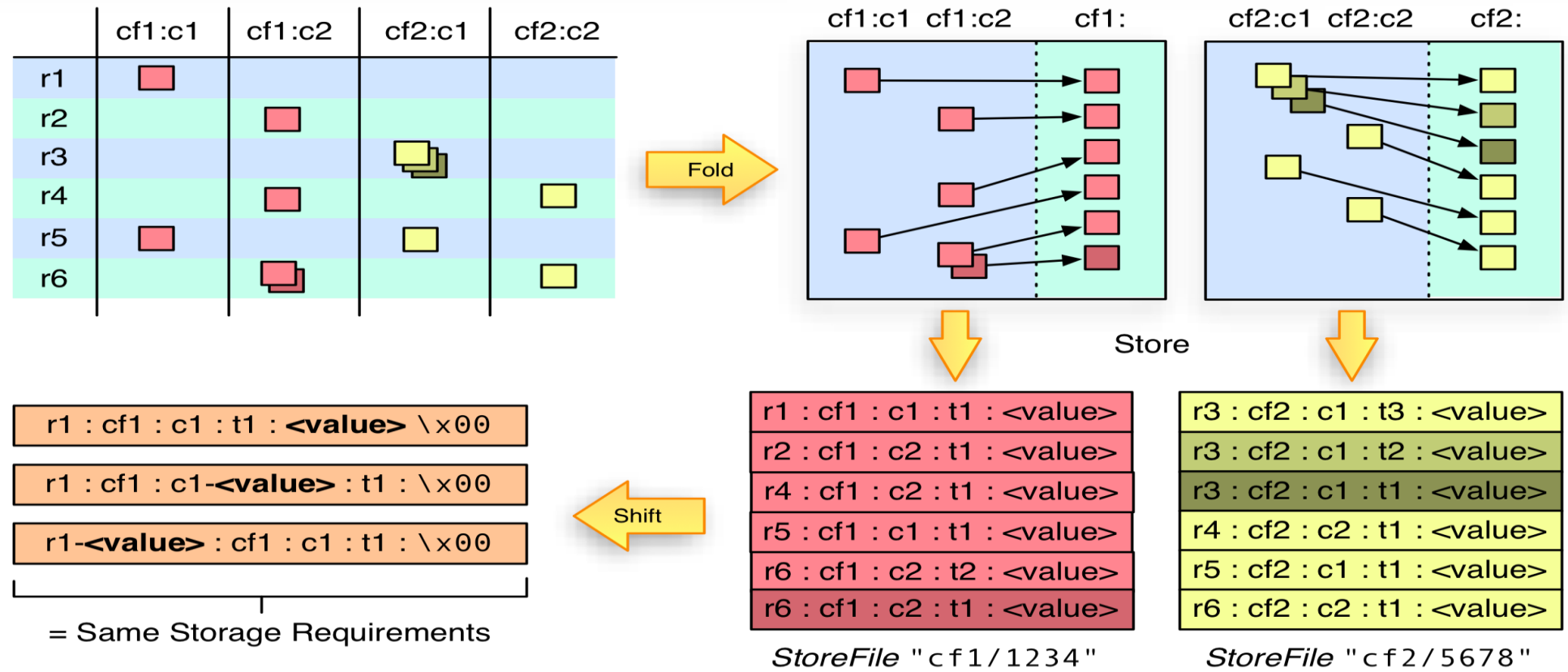
➤ HLog

- 含义：以WAL（Write Ahead Log，预写日志）方式写数据时产生的日志文件
- 目的：HRegionServer意外宕机时的数据恢复
- 先写HLog，再写MemStore，最后写StoreFile
- 每个HRegionServer维护一个HLog
- 定期删除HLog过期数据
- 数据恢复过程
 - HMaster通过ZooKeeper自动感知HRegionServer宕机
 - HMaster处理HLog，针对不同的Region拆分HLog
 - HMaster重新分配失效的Region
 - 通过HLog将尚未持久化的数据重新写入MemStore，然后溢写到StoreFile









RowKey升序
TimeStamp降序

➤ StoreFile Compaction

- 含义：将Store中的全部或部分StoreFile合并为一个StoreFile的过程
- 目的：减少StoreFile数量，提升数据读取效率
- 分类：
 - ① Minor Compaction（小范围合并）
 - 选取Store下的部分StoreFile（小的或相邻的），将它们合并为一个StoreFile
 - 不删除过期版本、delete marker数据
 - 速度较快，IO和带宽开销相对较低
 - ② Major Compaction
 - 将Store下的所有StoreFile合并为一个StoreFile
 - 删除过期版本、delete marker数据
 - 速度很慢，IO和带宽开销非常高

➤ StoreFile Compaction

• Minor Compaction

– 相关参数:

- `hbase.hstore.compaction.min`
- `hbase.hstore.compaction.max`
- `hbase.hstore.compaction.min.size`
- `hbase.hstore.compaction.max.size`
- `hbase.hstore.compaction.ratio`

• Major Compaction

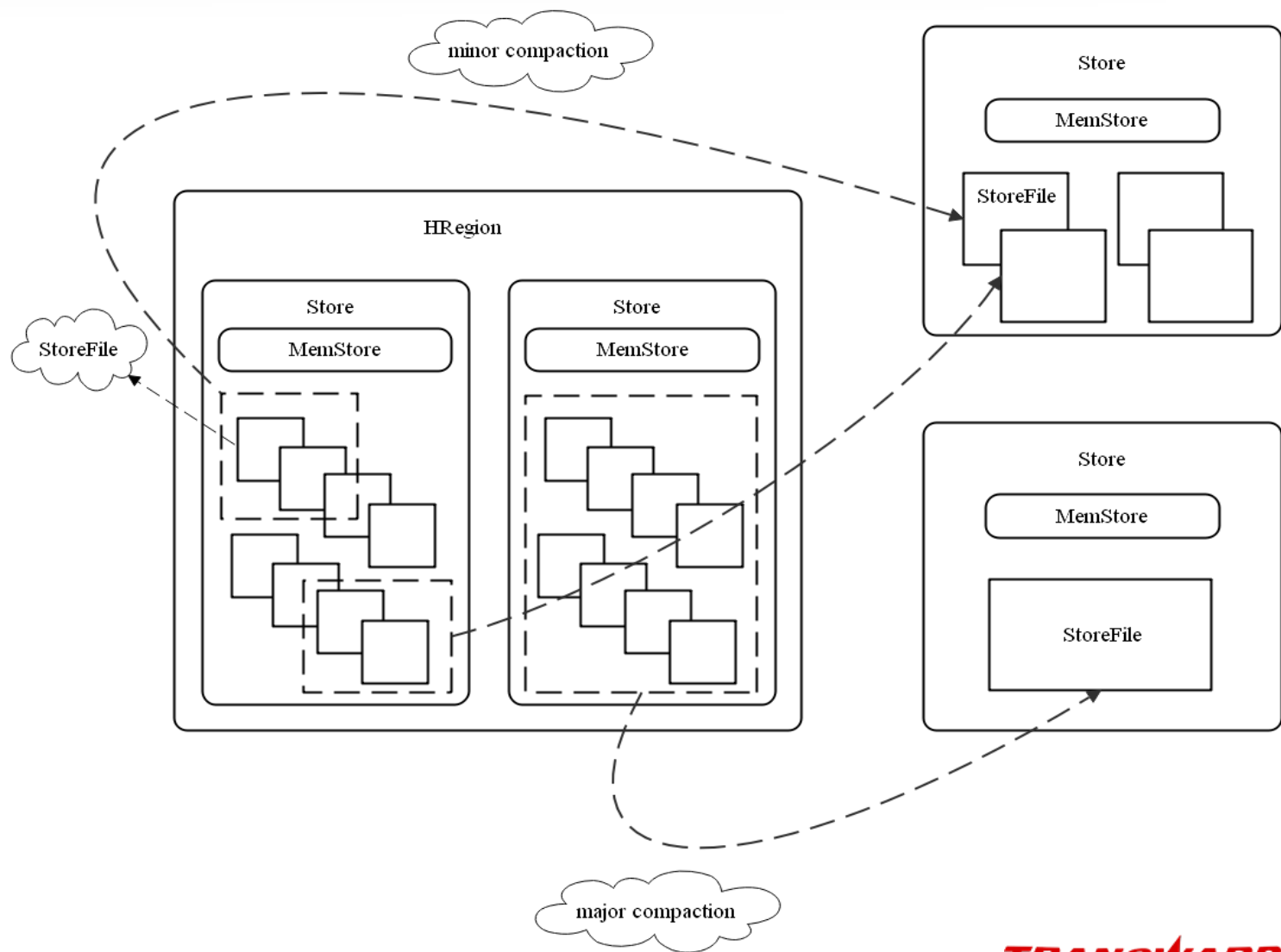
– 自动触发:

- `hbase.hregion.majorcompaction`
- `hbase.hregion.majorcompaction.jitter`

– 手动触发

- `major_compact` 命令
- API

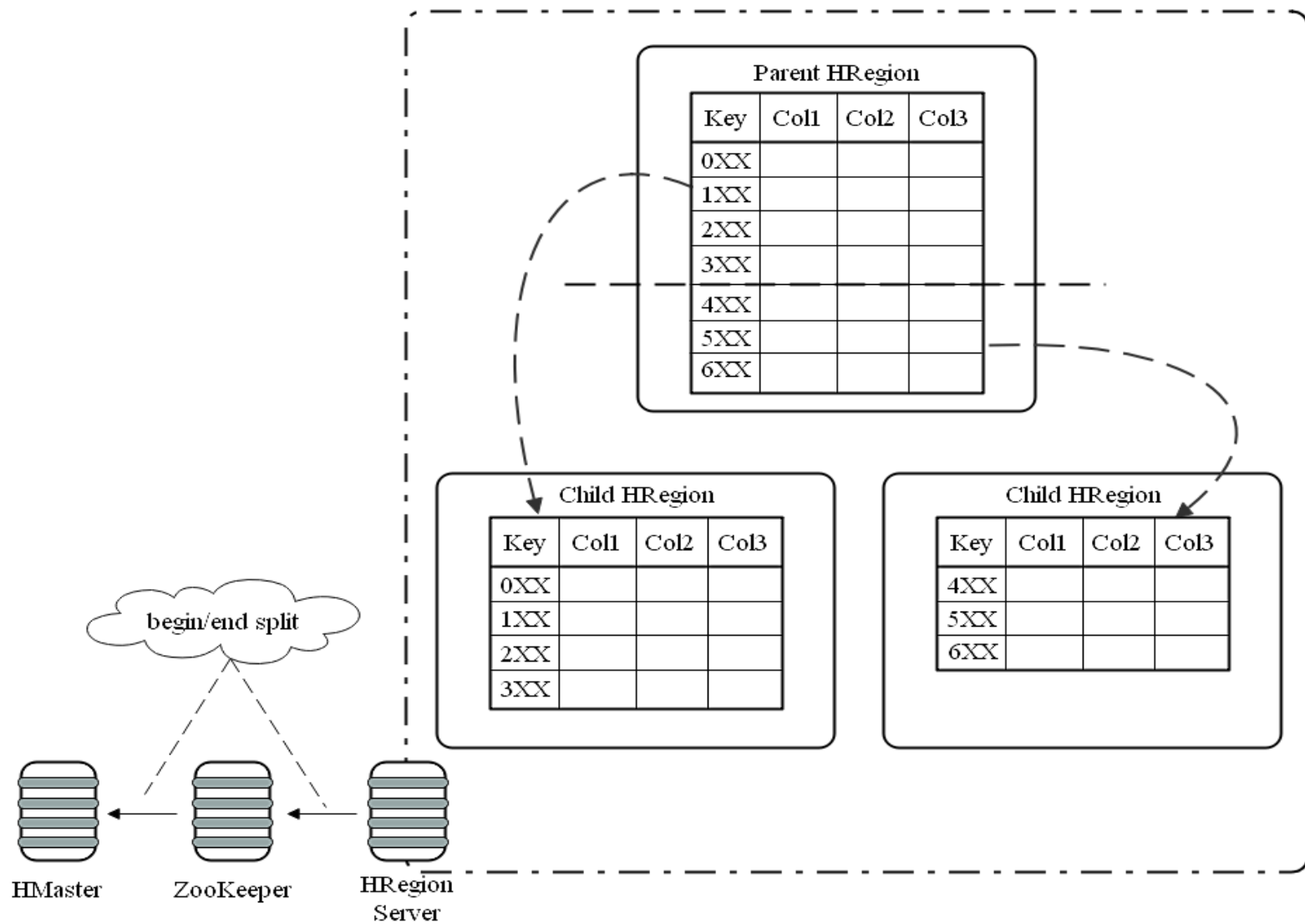
– 选择集群比较空闲的时间操作



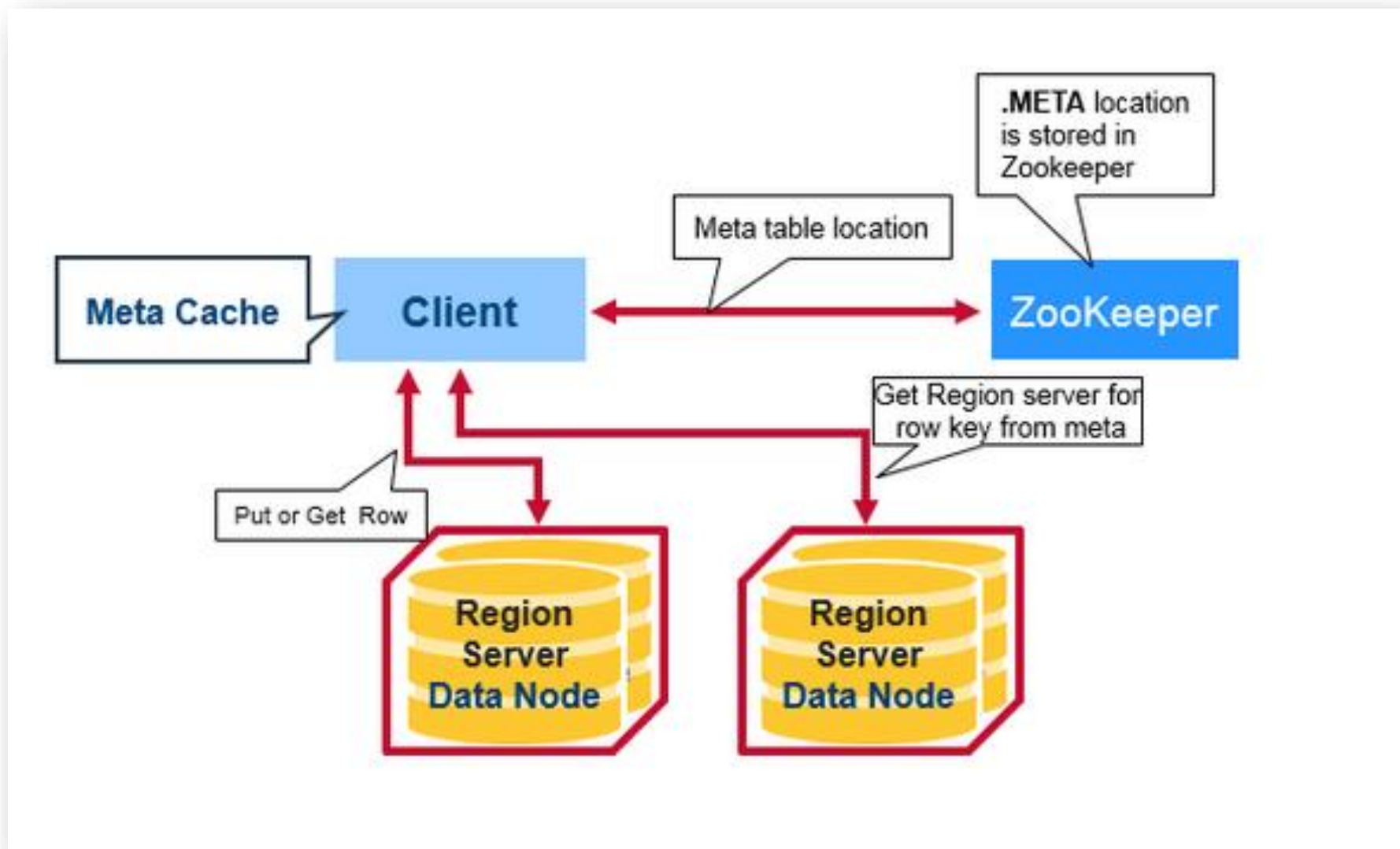
➤ Region Split

- 含义：根据一定的触发条件和分裂策略，将Region划分为两个子Region的过程
- 目的：实现数据访问的负载均衡
- 方法：利用Middle Key将当前Region划分为两个等分的子Region
- 触发
 - 条件：当Region中最大Store的大小超过阈值时，触发Region Split
- Split会产生大量的I/O操作
- Split开始前和Split完成后，HRegionServer都会通知HMaster
- Split完成后，HRegionServer会更新meta表

➤ Region Split



➤ 读写流程图



➤ 写过程

- (1) Client访问ZK，获取meta表所在Region的位置信息，并将该信息写入Client Cache
- (2) Client读取meta表，根据Namespace、表名和RowKey，获取将要写入Region的位置信息，并将meta表写入Client Cache
- (3) Client向HRegionServer发出写请求，HRegionServer先将操作和数据写入HLog（预写日志，Write Ahead Log，WAL），再将数据写入MemStore，并保持有序
- (4) 当MemStore的数据量超过阈值时，将数据溢写磁盘，生成一个StoreFile文件
- (5) 当StoreFile的数量超过阈值时，将若干小StoreFile合并（Compact）为一个大的StoreFile
- (6) 当Region中最大Store的大小超过阈值时，Region分裂（Split），等分成两个子Region

➤ 读过程

- (1) 获取将要读取Region的位置信息（见写过程的第1、2步）
- (2) Client向HRegionServer发出读请求
- (3) HRegionServer先从MemStore读取数据，如未找到，再从StoreFile中读取



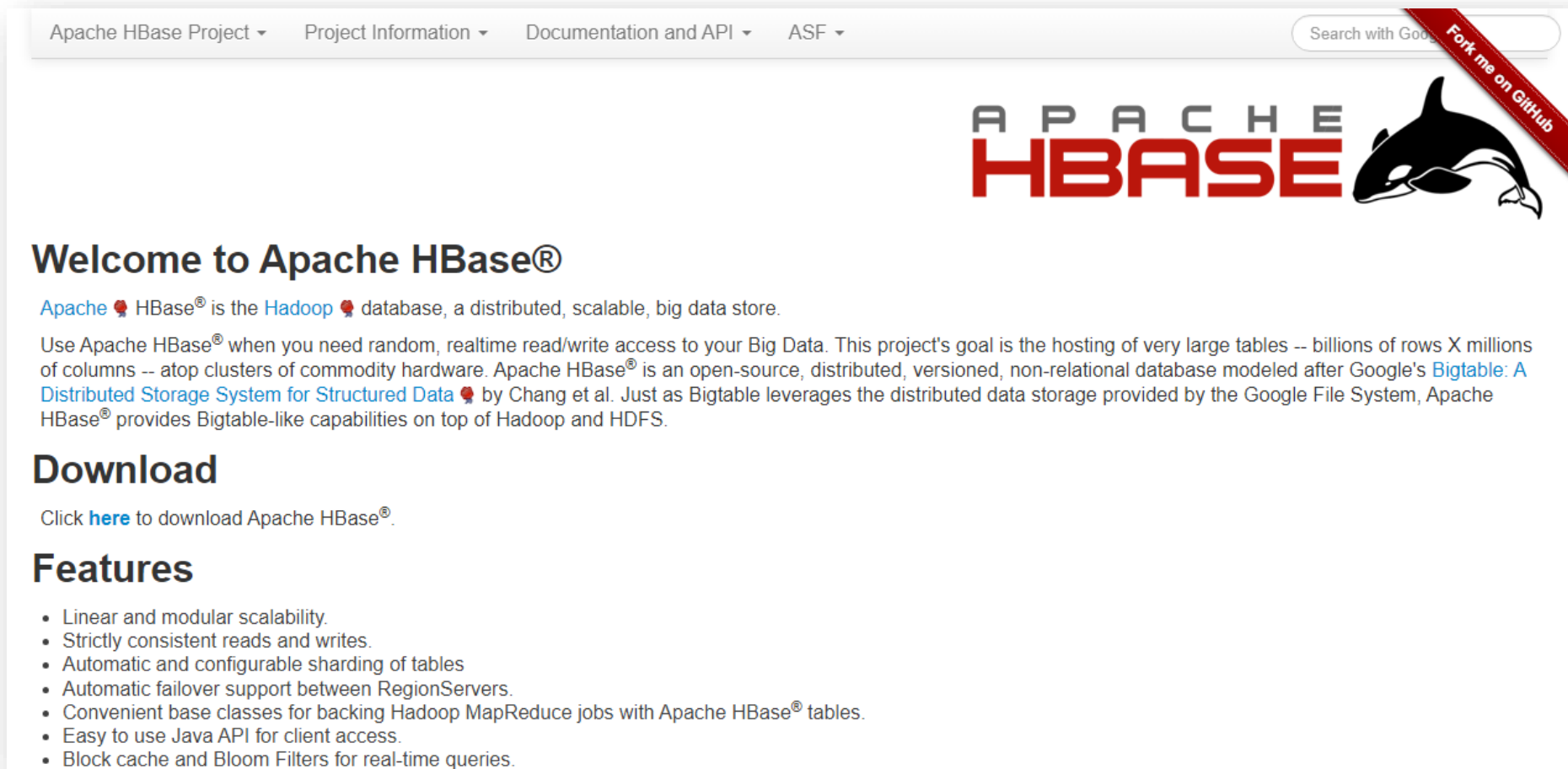
3

chapter

HBase部署

➤ 官网地址:

<https://hbase.apache.org/>





4 chapter

HBase基本用法

- ✓ HBase Shell
- ✓ HBase API

➤ 创建表

```
/* 创建表，cf1、cf2为列族名 */  
create 'table_name', 'cf1', 'cf2' ...
```

➤ 写入数据

```
/* 插入或更新数据，cf为列族名，cq为列名，value为要写入的数据 */  
put 'table_name', 'row_key', 'cf:cq', 'value'[, timestamp]
```

➤ 读取数据

```
/* 读取单行数据 */  
get 'table_name', 'row_key'  
/* 读取列族数据 */  
get 'table_name', 'row_key', 'cf'  
/* 读取列数据 */  
get 'table_name', 'row_key', 'cf:cq'
```

➤ 删除数据

```
/* 删除列 */  
delete 'table_name', 'row_key', 'cf: cq',[ timestamp]  
/* 删除行 */  
delete 'table_name', 'row_key'
```

➤ 全表扫描

```
/* 查看表的全部数据 */  
scan 'table_name'
```

➤ 计算表的行数

```
count 'table_name'
```

➤ HBase常用API示例:

```
Configuration conf = HBaseConfiguration.create();
conf.set("hbase.zookeeper.quorum", "192.168.1.100");
conf.set("hbase.zookeeper.property.clientPort", "2181");
HBaseAdmin admin = new HBaseAdmin(conf);

//判断表是否存在
if(isTableExist(tableName)){
    System.out.println("表" + tableName + "已存在");
}else{
    //创建表属性对象,表名需要转字节
    HTableDescriptor descriptor = new HTableDescriptor(tableName.valueOf(tableName));
    //创建列族
    descriptor.addFamily(new HColumnDescriptor(cf));
    //创建表
    admin.createTable(descriptor);
}

//禁用表
admin.disableTable(tableName);
//删除表
admin.deleteTable(tableName);
```


➤ HBase常用API示例:

```
//插入数据
//创建HTable对象
HTable hTable = new HTable(conf, tableName);
//向表中插入数据
Put put = new Put(Bytes.toBytes(rowKey));
//向Put对象中组装数据
put.add(Bytes.toBytes(columnFamily), Bytes.toBytes(column), Bytes.toBytes(value));
hTable.put(put);
hTable.close();

//查询单行数据
HTable table = new HTable(conf, tableName);
Get get = new Get(Bytes.toBytes(rowKey));
Result result = table.get(get);
for(Cell cell : result.rawCells()){
    System.out.println("行键:" + Bytes.toString(result.getRow()));
    System.out.println("列族" + Bytes.toString(CellUtil.cloneFamily(cell)));
    System.out.println("列:" + Bytes.toString(CellUtil.cloneQualifier(cell)));
    System.out.println("值:" + Bytes.toString(CellUtil.cloneValue(cell)));
    System.out.println("时间戳:" + cell.getTimestamp());
}
```

➤ HBase常用API示例:

```
//scan查询全表数据
Scan scan = new Scan();
//使用HTable得到resultcanner实现类的对象
ResultScanner resultScanner = hTable.getScanner(scan);
for(Result result : resultScanner){
    Cell[] cells = result.rawCells();
    for(Cell cell : cells){
        //分别查询到rowkey、列族、列名、值
        System.out.println("行键:" + Bytes.toString(CellUtil.cloneRow(cell)));
        System.out.println("列族" + Bytes.toString(CellUtil.cloneFamily(cell)));
        System.out.println("列:" + Bytes.toString(CellUtil.cloneQualifier(cell)));
        System.out.println("值:" + Bytes.toString(CellUtil.cloneValue(cell)));
    }
}
```

➤ HBase的rowkey设计：

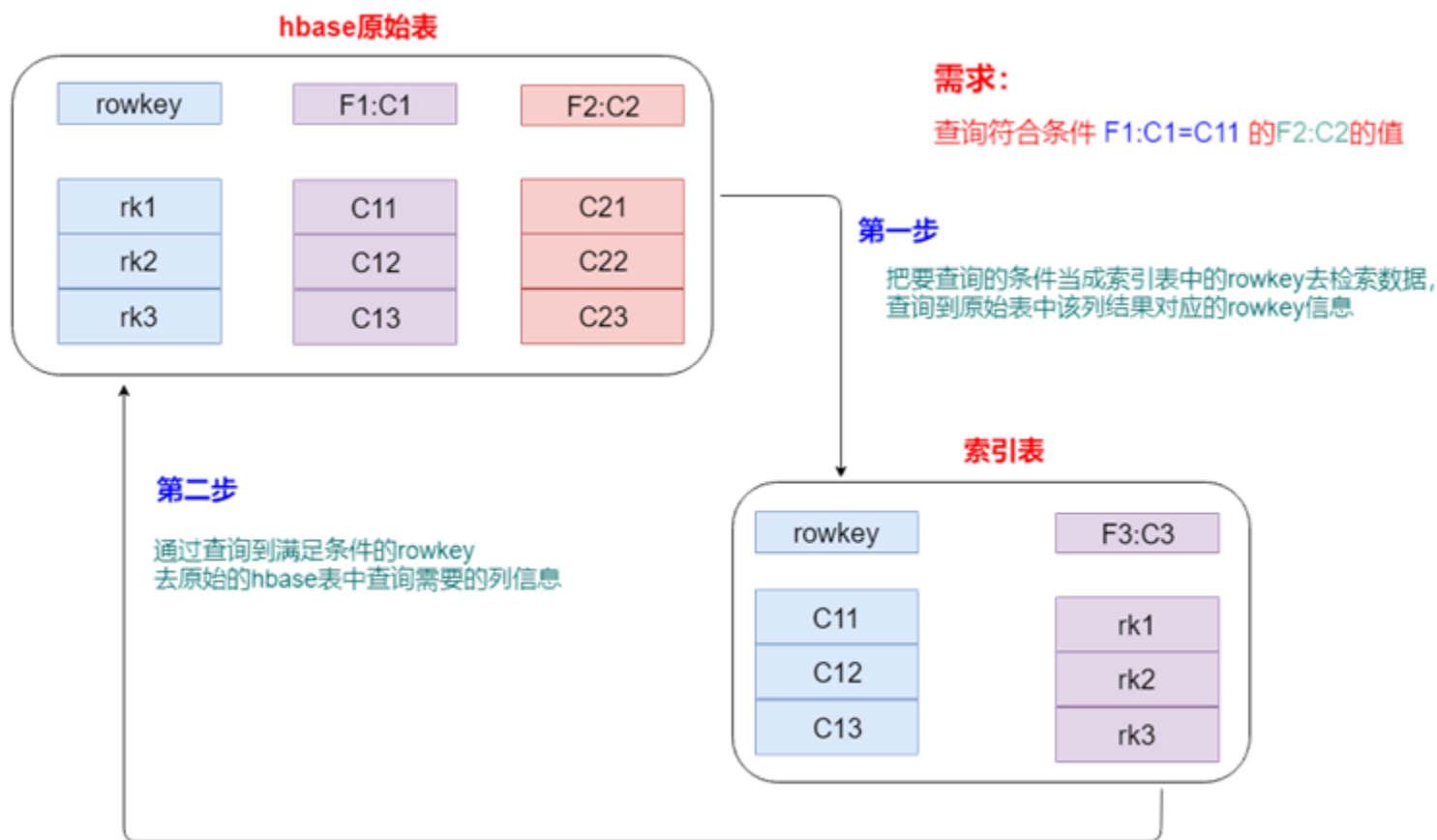
- 在满足业务要求的前提下，还需要考虑以下三个原则：

- ① 唯一原则，按照字典序排序
- ② 长度原则 —— 越小越好
- ③ 散列原则，避免出现**数据倾斜**



➤ HBase的二级索引:

- 以HBase Coprocessor (协处理器) 为例:
- 如果需要对于某一项Column建立二级索引, 就可以在Put、Delete的时候, 将其信息更新到另外一张索引表中。



➤ HBase & MapReduce:

```
Configuration config = HBaseConfiguration.create();
Job job = new Job(config, "ExampleSummary");
job.setJarByClass(MySummaryJob.class);      // class that contains mapper and reducer

Scan scan = new Scan();
scan.setCaching(500);                       // 1 is the default in Scan, which will be bad for MapReduce jobs
scan.setCacheBlocks(false);                 // don't set to true for MR jobs

TableMapReduceUtil.initTableMapperJob(
    sourceTable,                             // input table
    scan,                                    // Scan instance to control CF and attribute selection
    MyMapper.class,                          // mapper class
    Text.class,                             // mapper output key
    IntWritable.class,                      // mapper output value
    job);

TableMapReduceUtil.initTableReducerJob(
    targetTable,                             // output table
    MyTableReducer.class,                   // reducer class
    job);

job.setNumReduceTasks(1);                   // at least one, adjust as required

boolean b = job.waitForCompletion(true);
```

➤ HBase & MapReduce:

```
public static class MyMapper extends TableMapper<Text, IntWritable> {  
  
    public static final byte[] CF = "cf".getBytes();  
    public static final byte[] ATTR1 = "attr1".getBytes();  
  
    private final IntWritable ONE = new IntWritable(1);  
    private Text text = new Text();  
  
    public void map(ImmutableBytesWritable row, Result value, Context context) throws IOException, InterruptedException {  
  
        String val = new String(value.getValue(CF, ATTR1));  
        text.set(val); // we can only emit Writables...  
  
        context.write(text, ONE);  
    }  
}
```

➤ HBase & MapReduce:

```
public static class MyTableReducer extends TableReducer<Text, IntWritable, ImmutableBytesWritable> {  
  
    public static final byte[] CF = "cf".getBytes();  
    public static final byte[] COUNT = "count".getBytes();  
  
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {  
        int i = 0;  
  
        for (IntWritable val : values) {  
            i += val.get();  
        }  
  
        Put put = new Put(Bytes.toBytes(key.toString()));  
        put.add(CF, COUNT, Bytes.toBytes(i));  
  
        context.write(null, put);  
    }  
}
```




Q&A

TRANSWARP
星环科技